

CS106A Final Exam Reference Sheet

Graphics and Mouse functions/commands:

<code>canvas = Canvas(width, height, 'Name')</code>	Creates a new drawing canvas with a width of <code>width</code> pixels and a height of <code>height</code> pixels with a title of <code>Name</code>
<code>canvas.update()</code>	Updates the canvas to render the current position of all objects
<code>canvas.mainloop()</code>	Keeps the graphics window open/displaying until the window is closed.
<code>canvas.create_line(x1, y1, x2, y2, color)</code>	Draws a line from <code>(x1, y1)</code> to <code>(x2, y2)</code> in <code>color</code>
<code>canvas.create_rectangle(x1, y1, x2, y2, color)</code>	Draws a rectangle bounded by <code>(x1, y1)</code> and <code>(x2, y2)</code> in <code>color</code>
<code>canvas.create_oval(x1, y1, x2, y2, color)</code>	Draws an oval bounded by the rectangle <code>(x1, y1)</code> and <code>(x2, y2)</code> in <code>color</code> .
<code>canvas.create_text(x, y, anchor=anchor, font=font, font_size=font_size, text='some_text', color=color)</code>	Draws the text <code>text</code> at position <code>(x, y)</code> in <code>font</code> with <code>font_size</code> in <code>color</code> , anchored by <code>anchor</code> . <code>anchor</code> can be one of "n", "ne", "e", "se", "s", "sw", "w", "nw", or "center", that describes what side of your text you want to position at the <code>(x, y)</code>
<code>canvas.move(obj, dx, dy)</code>	Moves obj by adding <code>dx</code> to its current <code>x</code> value, and by adding <code>dy</code> to its current <code>y</code> value.
<code>canvas.moveto(obj, x, y)</code>	Moves obj to location <code>(x, y)</code>
<code>canvas.get_mouse_x()</code>	Returns <code>x</code> location of mouse
<code>canvas.get_mouse_y()</code>	Returns <code>y</code> location of mouse
<code>canvas.get_left_x(obj)</code>	Returns left-hand <code>x</code> location of obj
<code>canvas.get_top_y(obj)</code>	Returns top <code>y</code> location of obj
<code>canvas.get_obj_width(obj)</code>	Returns width of obj in pixels
<code>canvas.get_obj_height(obj)</code>	Returns height of obj in pixels
<code>canvas.find_overlapping(x_1, y_1, x_2, y_2)</code>	Returns a <i>list</i> of all objects that are overlapping the imaginary rectangle with upper-left corner at location <code>(x_1, y_1)</code> and bottom-right corner at location <code>(x_2, y_2)</code>

Time commands:

`time.sleep(seconds)`

Pauses the program for `seconds` number of seconds, which can be either an integer or decimal value.

Common Python List Functions:

<code>alist.append(x)</code>	Appends <code>x</code> to the end of <code>alist</code>
<code>alist.insert(index, x)</code>	Inserts <code>x</code> into <code>alist</code> at <code>index</code> After the insert function, <code>x</code> will be at <code>index</code>
<code>alist.pop()</code>	Removes the last element of <code>alist</code>
<code>alist.sort()</code>	Sorts the element of <code>alist</code> into ascending / alphabetical order
<code>len(alist)</code>	Returns the number of elements (length) of <code>alist</code>
<code>alist[index1:index2]</code>	Returns a slice (new list) from <code>index1</code> up to but not including <code>index2</code>

Common Python String Functions:

<code>str1 + str2</code>	Returns a new string which is the concatenation of <code>str1</code> with <code>str2</code>
<code>len(str)</code>	Returns the number of characters (length) of <code>str</code>
<code>str.strip()</code>	Returns a new string with leading/trailing spaces removed from <code>str</code>
<code>str.split()</code>	Returns a new list containing the terms from <code>str</code> where whitespace separates the terms in <code>str</code>
<code>str.find(string_to_find)</code>	Returns index of first occurrence of <code>string_to_find</code> in <code>str</code> . Returns -1 if <code>string_to_find</code> is not found in <code>str</code>
<code>str.isalpha()</code>	Returns <code>True</code> if all characters in <code>str</code> are letters in alphabet. Returns <code>False</code> otherwise.
<code>str.isdigit()</code>	Returns <code>True</code> if all characters in <code>str</code> are digits ('0' to '9'). Returns <code>False</code> otherwise.
<code>str.isspace()</code>	Returns <code>True</code> if all characters in <code>str</code> are whitespace (e.g., space, tab, newline). Returns <code>False</code> otherwise.
<code>ord(ch)</code>	Returns ASCII (ordinal) value of characters <code>ch</code>
<code>str[index1:index2]</code>	Returns a slice (new string) with characters from <code>str</code> starting at <code>index1</code> and going up to but not including <code>index2</code>

Sorting

sorted(alist)

Returns a new list that is the non-decreasing (increasing) sorted version of **alist**.

sorted(alist, reverse=True)

Returns a new list that is the non-increasing (decreasing) sorted version of **alist**. In other words, the returned list is in reverse sorted order.

sorted(alist, key=func)

Returns a new list that is sorted according to the function **func**.

Common Python File reading commands

To open a file, the following syntax is used:

```
with open(filename) as f:  
    # code using f here
```

Given an open file, **f**, the following functions can be used on **f**:

f.read()

Reads in the entire file, returning a string

f.readline()

Reads the next line in the file, returning a string

f.readlines()

Reads all of the lines in the file, returning a list of strings

for line in f:

Reads one line at a time from the file, where **line** will be a string that is the line read.

When all lines have been read, the loop ends.

Python Classes

Constructor

```
def __init__(self, ...):  
    body
```

Function (method) that is called when a new object of the class is created. Note that the ... represents 0 or more parameters that could be passed to the constructor when a new object is created.

Methods

```
def method_name(self, ...):  
    body
```

A function (method) that can be called on an object of the given class. Note that the ... represents 0 or more parameters that could be passed to the method when it is called on an object.

Instance Variables

Variables that are contained inside each object of the given class type. You refer to instance variables using the form:

```
self.variable_name
```